

classificatie_IMBD

October 21, 2025

1 IMBD Classificatie

In deze analyse train ik een machine learning model dat bepaalt of een filmreview positief of negatief is.

De dataset bestaat uit 50.000 reviews uit de IMDB-database, verdeeld in 25.000 trainings- en 25.000 testreviews.

```
[1]: import numpy as np
from sklearn.datasets import load_files
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn import svm
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
import re
import pickle
```

Reviews inladen

```
[2]: path_to_imdb = "C:/Users/lucas/Downloads/aclImdb_v1/aclImdb/train/"
#Alles uit de mappen neg en pos halen
movie_reviews_data = load_files(path_to_imdb, categories=['neg', 'pos'],
    ↪shuffle=True, random_state=42)
#De reviews
X_train = movie_reviews_data.data
#Lijst met neg of pos
y_train = movie_reviews_data.target
print(len(movie_reviews_data.data))
```

25000

1.1 Bag of Words

Als alle reviews zijn ingeladen, kunnen we de woorden omzetten naar kolommen

```
[3]: cv = CountVectorizer(  
      stop_words='english',  
      max_features=10000,  
      min_df=5,  
      max_df=0.9,  
      binary=True)
```

met `stop_words = english` haalt hij alle woorden zoals `the`, `and` enzovoorts uit de tekst. Dit is ook niet belangrijk om te weten of iets positief of negatief is. Met `max_features=10000` behouden we alleen de 10000 meest gebruikte woorden. Met `min_df` halen we woorden weg die minder dan 5 keer voorkomen. Dit is handig om zeldzame woorden en ruis weg te halen. Met `max_df` halen we de woorden weg die in meer dan 90% van de teksten voorkomen. Met `binary = true` zeggen we alleen dat een woord voorkomt in een tekst, maar niet hoe vaak. Dit werkte beter voor de performance.

Nu kunnen we de 10000 woorden gaan uitzoeken

```
[4]: cv.fit(X_train)
```

```
[4]: CountVectorizer(binary=True, max_df=0.9, max_features=10000, min_df=5,  
      stop_words='english')
```

Daarna kunnen we gaan kijken hoe vaak elk woord voorkomt per review

```
[5]: X_train_bow = cv.transform(X_train)
```

Nu kunnen we een model gaan uitkiezen om te gebruiken

```
[6]: # clf = CategoricalNB() Snel maar niet heel goed  
      clf = LogisticRegression(  
          random_state=0,  
          max_iter = 1000,  
      ) #snel en prima  
      # clf = RandomForestClassifier(max_depth=2, random_state=0) Slecht en traag  
      # clf = MLPClassifier(random_state=1, max_iter=300) Laad niet eens volgens mij  
      ↪wel goed  
      # clf = svm.SVC() Laad niet eens volgens mij ook goed
```

Na wat testen werkte voor mij `LogisticRegression` het beste. `CategoricalNB` was super snel, maar de accuracy was niet heel hoog. `RandomForestClassifier` was erg traag en had een lage accuracy. `MLPClassifier` en `svm.SVC` laadden bij mij helemaal niet na 10 minuten wachten. Daarom heb ik gekozen voor `LogisticRegression`. Dit was redelijk snel en had een goede accuracy.

Nu kunnen we het model gaan trainen op basis van de trainingsdata. Hij probeert de uitkomst van de neg en pos zo goed mogelijk te voorspellen, door gewichten te geven aan de woorden. Een woord als bijvoorbeeld `'great'` telt dan veel mee om te kijken of een review positief of negatief is.

```
[7]: clf.fit(X_train_bow, y_train)
```

```
[7]: LogisticRegression(max_iter=1000, random_state=0)
```

Nu gaan we ipv de trainingsdata, de echte testdata ophalen

```
[8]: path_to_imdb = "C:/Users/lucas/Downloads/aclImdb_v1/aclImdb/test/"
      #Data inladen uit de test folder
      movie_reviews_data = load_files(path_to_imdb, categories=['neg', 'pos'],
      ↪shuffle=True, random_state=42)
      #Reviews laden uit de neg en pos mappen
      X_test = movie_reviews_data.data
      #Reviews
      y_test = movie_reviews_data.target
      #Label met pos of neg
```

De volgende stap is om weer te kijken hoe vaak elk van de 10000 woorden voorkomt per review(omzetten naar tabel met 0 en 1en)

```
[9]: X_test_cv = cv.transform(X_test)
```

Nu kunnen we een label(pos of neg) voorspellen voor elke review

```
[10]: predicted = clf.predict(X_test_cv)
```

Nu dat is gedaan, kunnen we de labels uit y_test gaan vergelijken met wat het model heeft voorspeld

```
[11]: acc = accuracy_score(y_test, predicted)
      print(acc)
```

0.8558

Na heel wat aanpassen van de max_features, trainingsmodellen en andere variabelen is dit het beste eindresultaat dat ik kon krijgen(85.6%).

Nu de vectorizer dumpen met pickle

```
[13]: with open('vectorizer.pkl', 'wb') as f:
      pickle.dump(cv, f)
```

En met model zelf ook

```
[14]: with open('model.pkl', 'wb') as f:
      clf = pickle.dump(clf, f)
```

```
[ ]:
```